Problem Set: Text and Images as Data, and Claim Graphs

Language Modelling for Economics, ECB DG Research, 19–20 November 2026

The problem set has two parts. Work in groups of two to three. You are encouraged to use AI coding agents throughout; document how you used them and how you checked their output.

Part 1: Text and images as data

Data

Use the dataset provided with this problem set, `data/news_videos.csv.gz`. It contains 23,689 YouTube videos posted by two US cable news outlets, Fox News (11,540) and MSNBC (12,149), from January to August 2026, one row per video:

- `video_id`, `url`: identifier and link
- `outlet`, `channel_id`: which outlet posted it
- `title`, `description`: the text
- `published_at`: date and time of posting
- `view_count`, `like_count`, `comment_count`: engagement
- `duration_seconds`: video length
- `days_since_publication`: how long the video has been online
- `thumbnail_url`: the image the outlet chose to represent the video, fetched live from the URL

Both outlets post 1,350–1,700 videos a month, so the panel is close to balanced in every month. Views accumulate, so control for `days_since_publication` whenever engagement is an outcome. `pandas.read_csv` reads the gzipped file directly. Embedding all 23,689 thumbnails is slow on a laptop or free Colab GPU: start with a random subsample of a few thousand. `data/build_dataset.py` rebuilds or extends the dataset with a free YouTube Data API key.

If you prefer, you may work on your own text or image data instead.

Questions

1. **Representations.** Represent the titles and descriptions as vectors (bag-of-words, Word2Vec, BERT, sentence transformers, or another method; explain your choice). Represent a subsample of thumbnails as vectors with an image encoder, preferably one aligned with language such as CLIP. Record the exact model versions.

2. **Clustering.** Cluster the text and image representations (PCA, k-means, and so on). Can you name the clusters after reading the videos nearest each centroid? Do the outlets differ in their cluster shares, or over time?

3. **Prediction.** Train a classifier (logistic regression, random forest, gradient boosting) to:

   a. predict the outlet from the thumbnail alone, reporting held-out accuracy and AUC;
   b. predict engagement, modelled as log(1 + views), from text, images, and both, controlling for `days_since_publication` and `duration_seconds`.

   Compare text-only, image-only, and joint models. **Warning:** outlets brand their thumbnails with logos, chyrons, and studio sets. A classifier with AUC 0.99 has probably learned to find a logo, not a political style. Inspect the images the model is most and least confident about, and check whether performance survives cropping or masking the branded region.

4. **Zero-shot image measures.** Because CLIP puts images and text in one space, score thumbnails against short descriptions (for example, "a photo of an angry crowd") to build a continuous measure of image content without training data. Report the prompt and check that your result survives reasonable rewordings.

5. **Labelling with an LLM (required).** Using a free model through the OpenRouter API, label a reproducible random sample of 40 videos, with the title and description as the document.
    a. Define one construct (for example, primary topic, tone, or presence of conflict), with clear labels and an uncertain/other option.
    b. Write one fixed prompt asking for exactly one label. Save the prompt, the requested model, and the model reported in the API response. Do not put your API key in the notebook.
    c. Label at least 15 of the same documents by hand. Report agreement, inspect the disagreements, and explain one recurring source of error.
    d. Use the model labels for one simple comparison, such as topic shares by outlet. Say what it suggests and why the labels still require validation.
6. **Optional: a webpage.** Use an AI coding agent to turn one finding into a small interactive webpage (a "guess the outlet" game, a thumbnail gallery, a cluster explorer). Check that it represents your analysis accurately.

Off-the-shelf models may be unreliable on tasks they were not built for. Validate their output on a subsample you have looked at yourself.

Part 2: Extending claim graphs beyond economic theory

Background

Ash, Ciccarone, and Galletta (2026), "Mapping Economic Theory with AI," represent a theory paper as a **claim graph**: a directed hypergraph whose nodes are statements (assumptions, definitions, lemmas, propositions, economic claims) and whose hyperedges are derivations, each deriving one target from a set of sources used jointly. Several derivations into the same target are alternatives. On this structure they define descriptive measures (yield, depth, fan-in, coverage, redundancy) and an intervention measure, **network fragility**: the weighted share of headline and major claims that lose support when a premise or derivation is removed.

The repository's `code/claim_graph.py` implements these measures and a one-call LLM extraction step.

Task

Extend this approach to a different kind of economic or financial text, where conclusions also rest on chains of premises. Suggested domains (or choose your own):

1. **Monetary policy communication.** Map an ECB monetary policy statement or account of a Governing Council meeting: which projections, risk assessments, and judgements support the policy decision and the forward guidance?
2. **Financial stability reports.** Map the argument from indicators and stress-test results to the stated vulnerabilities and policy recommendations.
3. **Empirical papers.** Map the argument from data, identifying assumptions, and estimates to the headline causal claims. Which identifying assumption carries the most weight?
4. **Equity research or credit rating reports.** Map how forecasts and assumptions support a valuation, rating, or recommendation.
5. **Legal and regulatory texts.** Map a court ruling on central bank powers or a supervisory decision from legal premises to holdings.

Questions

1. **Representation (design).** Adapt the representation to your domain. What are the node types (replacing or extending the paper's 23 types)? What counts as a "headline" claim, and what counts as a premise? Is the AND/OR hyperedge structure appropriate, or does your domain need something else (for example, probabilistic or weighted support, or evidence that strengthens rather than establishes a claim)? Justify each choice in a few sentences.

2. **Extraction.** Modify the extraction prompt in `claim_graph.py` for your domain and apply it to at least three documents. Save each graph as JSON.
3. **Validation.** For one document, build the graph by hand (or carefully correct the LLM's graph) and compare it with the automated version. Report agreement on nodes, on headline claims, and on edges. Where does the model go wrong?
4. **Measurement.** Compute the structural measures and fragility for each graph. Which premise has the largest load? Interpret one result substantively: what would it mean for the document's conclusions if that premise were wrong?
5. **An economic question.** Use your measures to say something about your domain. Examples: do policy statements become more or less fragile over time, or in crises? Do empirical papers with more robustness checks have higher route redundancy? Do rating changes rest on fewer premises than affirmations? With only a handful of documents, state this as a well-motivated hypothesis and describe the data and design you would need to test it.
6. **Limits.** In half a page, discuss what the graph does not capture in your domain, and the main risks of relying on LLM-extracted graphs for policy or investment decisions.

Deliverables

- A short write-up (at most six pages) covering Part 1 and Part 2.
- Your code, notebooks, and JSON graphs in a repository or zip file.